

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 3.1]

MESSAGE PASSING MODELS

SUMMARY

- Message passing communication model
 - basic primitives
 - synchronous/asynchronous
 - patterns
 - client-server interactions
 - active resource managers (vs monitors)
 - guarded communication
 - peer-to-peer interaction

il componente attivo quando chiama un metodo di un componente passivo (entra nel monitor) trasferisce il proprio controllo al Monitor che manda in esecuzione una propria entry con il flusso di controllo del componente attivo



MESSAGE PASSING

-  Rispetto al OO, cambia che non c'è trasferimento del flusso di controllo (incapsulamento del flusso di controllo)

Basic idea

- the only way for processes to interact is by exchanging messages
 - **send and receive** primitives 
- no other abstractions are allowed 
 - no shared objects, no monitors, no locks, no semaphores, etc
- Natural model for *distributed programming*, but more and more considered also for concurrent programming in general in mainstream languages / frameworks / technologies
 - to avoid pitfalls introduced by multi-threaded programming & related synchronization & mutual exclusion mechanism
 - implemented in HTML5 Web Workers, actors frameworks, recent languages (Go, DART,...)

Sostanzialmente, due primitive (send and receive) e l'astrazione di prima classe di canale (astrazione di prima classe significa elemento del linguaggio che può essere manipolato con la stessa libertà di un dato primitivo)

inviare un messaggio, senza trasferire il controllo, da un componente attivo ad un altro (metto un messaggio sul canale) e ricevere un messaggio (ricevo un messaggio sul canale definito)

(non c'è più memoria condivisa tra componenti attivi, quindi comunicano tramite scambio di messaggi, cioè trasferimento di dati, non del controllo come avveniva con i Monitor che quando si chiamava un suo Metodo si trasferiva il controllo, violando l'incapsulamento del flusso di controllo)

ORIGIN OF THE MODEL

- Asynchronous message passing introduced by Brinch-Hansen in 1970
 - designing Operating Systems for the Danish RC4000 computer
- Bob Balzer in 1971 introduced the notion of communication port, which is at the base of modern async message passing
- Synchronous communication introduced by C.A.R. Hoare in 1978
 - CSP formalism
- In 1973 Carl Hewitt and the following years Gul Agha (Carl's PhD student) introduce the actor model
 - based on asynchronous message passing
- Until recently, message passing has been explored almost in OS design & implementation
- With the multi-core and many-code, the model is getting a momentum as an alternative model to classic multi-threaded programming

BASIC COMMUNICATION PRIMITIVES

- Channel type

- **chan** ch(type id1, ..., type idn)

Esempio: `chan request(int clientID, int opCode);`
Questo definisce un canale chiamato `request` in cui ogni messaggio è composto da due valori interi: l'identificativo del client e il codice dell'operazione richiesta.

- `ch` is the channel name

- `type` and `id` are the types and the names of the data fields in messages transmitted via the channel

Definisce la struttura del messaggio. Ogni messaggio che passa per questo canale è una sequenza di uno o più campi tipizzati

- Communication primitives:

- **send** ch(expr1, expr2, ...)

A differenza della chiamata a un metodo in OOP (o in un Monitor), l'esecuzione di una `send` non trasferisce il controllo all'altro processo: invia semplicemente un dato.

- sending a message composed by expressions `expr1`, `expr2`, ... through the channel `ch`

- `expr` are expressions whose type must be the same as those of the corresponding fields in the declaration of `ch`

- **receive** ch(var1, var2, ... varn)

- receiving a message from a channel `ch`

- `var1`, ... are variables whose type must be the same as those of the corresponding fields in the declaration of `ch`



Il processo ricevente specifica una lista di variabili locali (`var1`, `var2`, ...). Quando un messaggio viene prelevato dal canale, i singoli campi del messaggio vengono assegnati a tali variabili in ordine.
Tipizzazione: Il tipo delle variabili deve corrispondere a quello dichiarato nella struttura del canale.

ATOMICITY

Quando parliamo di atomicità nell'accesso al contenuto di un canale, intendiamo che le primitive di invio (send) e ricezione (receive) vengono eseguite dal sistema come primitive atomiche.

- The access to the content of each channel is atomic
 - channels are declared global to processes



L'atomicità riguarda solamente la struttura dati interna del canale (la sua coda FIFO, i suoi puntatori di lettura/scrittura e i messaggi contenuti). Il resto del programma continua a girare concorrentemente. L'atomicità del canale non crea una sezione critica globale e non congela l'esecuzione degli altri processi o thread: Altri processi possono continuare a fare calcoli, accedere alla propria memoria locale o comunicare su altri canali.

SYNCHRONOUS VS. ASYNCHRONOUS COMMUNICATION

Tipo di Scambio di messaggi

↪ C'è un punto di sincronizzazione tra send e receive (si aspetta che entrambi sono in ascolto sul canale)

• Synchronous communication

Utilizzato in quelle architetture in cui i canali non avevano il Buffer e quindi si era obbligati ad aspettare che il messaggio sia arrivato all'altro componente attivo

- a send of a msg on a channel is blocked until the message is received on the same channel
- a receive is blocked until a message is inserted in the channel
- most primitive model, used in process algebra

il componente attivo che fa la send si blocca aspettando che l'altro componente attivo riceva il messaggio (questo perché non c'è una memoria di appoggio dove lasciare e continuare con il proprio flusso di controllo). Così facendo sono sicuro che quando mi sblocco il messaggio è stato ricevuto

• Asynchronous communication

In questo caso invece non c'è sincronizzazione perché c'è un Buffer che memorizza i messaggi del sender che poi il receiver usa il Buffer per prenderli in modo asincrono

- channels have a FIFO buffer where messages are enqueue
- a send of a msg on a channel succeeds as soon as the message is enqueued in the channel
- a receive is blocked until a message is available in the channel
- in some models/systems, channels with asynchronous message passing are also called ports

COMMUNICATION SCHEMES

Iniziare a considerare che i Canali sono usabili come Variabili/Oggetti molto agili, che creo ed uso in modo dinamico.

Gli schemi (o topologie) di comunicazione definiscono la struttura delle relazioni tra i processi, determinando quanti mittenti e quanti destinatari possono condividere ed eseguire operazioni (send e receive) sullo stesso canale.

- • **Simplest: one-to-one** communication model
 - a channel can be used only by a couple of processes
 - typical in synchronous communication
 - e.g. Occam language, implemented on Transputers
- • **Most general: many-to-many**
 - the same channel can be used by multiple senders and multiple receivers
 - competition in receiving messages
 - non-determinism
- • **many-one** scheme
 - typically used with ports
 - = each port can have only a single receiver, but multiple senders

Multipli produttori per un solo Consumatore (non c'è competizione, il consumatore vede tutti i messaggi). Utile per richiesta da client a server (request dal client e reply dal server: client fanno send sul canale request (con content e ID client) e ricevono la risposta sul proprio canale (creato grazie a ID client) riguardante le risposte (questo perché ogni canale è unidirezionale ed ha un proprio scopo))

Perché non usare un unico canale di risposta condiviso?

La receive standard preleva semplicemente il primo messaggio disponibile, senza possibilità di filtrarlo per destinatario. Con un unico canale di risposta condiviso tra tutti i client, si genererebbe una competizione errata (race condition): un client potrebbe sottrarre la risposta destinata a un altro. Assegnare un canale dedicato a ogni client (es. reply[clientID]) garantisce che ciascuno legga solo la propria risposta.

→ • one-to-many: un solo produttore per multipli consumatori (competizione tra chi fa receive: solo uno lo riceve). Utile per assegnare un task ad uno solo dei thread disponibili

SIMPLE EXAMPLES

P manda parallelamente una richiesta sia a Q sia a R (Q ed R hanno un proprio canale su cui ricevono le richieste) e poi si mette in attesa di ricevere le risposte da Q ed R: la richiesta e risposta vanno nello stesso canale perchè c'è solo un sender e receiver per ogni risposta/richiesta

```
chan requestA(int value)
chan requestB(int value)
chan response(int value)
```

3 canali aperti

P

```
r1,r2: integer
...
send requestA(5)
send requestB(6)
receive response(r1)
receive response(r2)
write(r1+r2)
...
```

Q

```
r: integer
...
receive requestA(r)
send response(r*2)
...
```

R

```
r: integer
...
receive requestB(r)
send response(r+1)
...
```

Due receive sullo stesso canale e poi stampa r1+r2. In questo caso, non mi interessa l'ordine con cui ricevo indietro le risposte (nel caso, avessi dovuto fare una divisione, l'ordine con cui ricevo le risposte è importante: avrei dovuto mettere in campo due canali al posto di uno solo così da garantire l'ordine)

SIMPLE PRODUCER-CONSUMER

- A producer and a consumer interacting by means of a single channel

channel buf(int)	
PRODUCER	CONSUMER
<pre>integer x loop forever: p1: x ← produce p2: send buf(x)</pre>	<pre>integer y loop forever: q1: receive buf(y) q2: consume(y)</pre>

A PIPELINE

- A process assembling lines of characters
 - functioning as a *filter* in pipeline architecture

Linguaggio simile al C dove sono stati introdotti i canali

Questo processo è uno stadio di un'architettura Pipeline che riceve dei caratteri sul canale, li colleziona in un buffer in locale, e, non appena c'è il CR (carriage return) oppure i caratteri che ha ricevuto hanno raggiunto una dimensione massima, stacca ed invia la stringa come messaggio sul canale di output

```
chan input(char), output(char[MAXLINE])

process CharToLine {
    char line[MAXLINE+1];
    int i = 0;
    while (true) {
        receive input(line[i]);
        while (line[i] != CR and i < MAXLINE) {
            i = i + 1;
            receive input(line[i]);
        }
        line[i] = EOL;
        send output(line)
        i = 0;
    }
}
```

processo che vuole in input un canale di caratteri (stream di caratteri che arriva in modo asincrono) ed in output produce una stringa

Processo sempre attivo

Il processo è bloccante (Sincronizzazione)
Quando il codice dice receive input(line[i]), il processo CharToLine si ferma. Non fa più nulla. Resta lì "congelato" finché qualcuno non gli invia un carattere.
Perché? Perché non può processare dati che non ha ancora ricevuto.
Lo stesso vale per la send: in molti modelli di Message Passing (come quello sincrono), il mittente si blocca finché il destinatario non ha ricevuto il messaggio.

Il sistema è concorrente (Esecuzione)
Mentre CharToLine è bloccato in attesa di un carattere, gli altri processi della pipeline non lo sono.
Il processo "a monte" (quello che genera i caratteri) sta lavorando per produrre il prossimo dato.
Il processo "a valle" (quello che riceve la riga completa) sta magari stampando o analizzando la riga precedente.

CLIENT-SERVER INTERACTION

- General client-server architecture using channels

```
chan  request(int,kind,arg_type);
chan[NCLIENTS]  reply(arg_result);

process Client[i = 0...N-1] {
    arg_type myargs;
    res_type myres;

    <init args>
    send request(i,opXXX, myargs);
    receive reply[i](myres);

}
```

SERVER SIDE: ACTIVE-MONITORS

- Server process ^{operations} waiting request ~~op~~ on a request channel and providing a result on a *dedicated* channel
 - functioning as **active monitor** Server simile ad un Monitor
 - typically used for implementing ***resource allocator processes***

```
chan request(int,kind,arg_type);
chan[NCLIENTS] reply(arg_result);

process Server {
    int clientID; op_kind kind;
    arg_type args; res_type res;

    <init code>
    while (true) {
        receive request(clientID,kind,args);
        if (kind == op1){
            body of op1
        } else ...
        } else if (kind == opN){
            body of opN
        }
        send reply[clientID](results);
    }
}
```

PASSIVE AND ACTIVE RESOURCE ALLOCATORS

Active Resource Allocators

```
chan request(int cliendID,int type)
chan[N] reply(res_id id)
```

Traduzione Implementazione
di un Monitor

```
process ResAllocator {
    int clientID;
    int avail = MAXUNITS;
    queue pending; # initially empty
    set units = <initial value>;
    op_kind kind; res_id id;

    while (true) {
        receive request(clientID,kind);
        if (kind == ACQUIRE){
            if (avail > 0)
                avail = avail - 1;
            id = remove(units);
            send reply[clientID](id);
        } else insert(pending,clientID);
    } else if (kind == RELEASE){
        if empty(pending){
            insert(units,id);
            avail = avail + 1;
        } else {
            remove(pending,clientID);
            send reply[clientID](id);
        }
    }
}
```

Può capitare che l'allocatore non abbia le risorse disponibili per la richiesta e quindi non può fare subito la reply (server invia la risposta solo quando è in grado di farlo, cioè quando ha la quantità delle risorse richieste). Essendo che non posso usare Wait, dato che non è possibile bloccare il flusso di controllo del richiedente, semplicemente metto in queue la richiesta ricevuta dal richiedente. Quindi, la send posso farlo quando è il momento più opportuno (quando ho nuove risorse e ci sono richiedenti in pending sul canale request)

```
process Client[i = 0...N-1] {
    arg_type myargs; res_id id;
    ...
    send request(i,ACQUIRE);
    receive reply[i](id);
    #...use resource
    send request(i,RELEASE);
}
```

Message Pa

Passive Resource Allocators (monitor)

```
monitor ResAllocator {
    int avail = MAXUNITS;
    set units = <initial value>;
    cond free;

    procedure res_id acquire(){
        res_id id
        if (avail == 0)
            waitC(free);
        else
            avail = avail - 1;
        id = remove(units);
        return id
    }

    procedure release(res_id id){
        insert(units,id);
        if (emptyC(free))
            avail = avail + 1;
        else
            signalC(free);
    }
}
```

Nei Monitor, si utilizzano le condition variable per bloccare un Thread facendolo aspettare di ricevere la risorsa richiesta (viene bloccato un flusso di controllo, quello del Thread richiedente la risorsa)

PASSIVE AND ACTIVE RESOURCE ALLOCATORS

Monitor-Based	Process-Based
Permanent variables	Local server variables
Procedure identifiers	request channel and op kinds
Procedure call	<code>send request(...)</code> <code>receive reply(...)</code> Il client invia un messaggio di richiesta sul canale del Server e subito dopo si mette in attesa bloccante per la risposta sul proprio canale privato
Monitor entry Ingresso / Mutua Esclusione (un unico processo alla volta dentro il Monitor)	<code>receive request(...)</code> La mutua esclusione è naturale e intrinseca: il Server è un singolo thread che estrae ed elabora un messaggio alla volta in modo strettamente sequenziale dal canale request.
Procedure return	<code>send reply(...)</code> Il Server invia un messaggio sul canale di risposta dedicato del client, permettendo a quest'ultimo di sbloccarsi e proseguire.
<code>wait</code> statement	Save pending requests
<code>signal</code> statement	Quando il Server gestisce un rilascio, controlla la propria lista locale pending, preleva il primo client in attesa e finalmente gli invia la <code>send reply(...)</code> con la risorsa. Retrieve and process pending reqs
Procedure bodies	cases in switch statement on op kind

GUARDED COMMUNICATION

- Come faccio a fare una Receive su più canali? (cioè come permettere a un processo di gestire più canali contemporaneamente senza rimanere bloccato su uno solo di essi.)
- Problem: *how to receive messages that can arrive on multiple channels at the same time?* Perchè se faccio una Receive su un canale, mi blocco se non c'è la risposta (posso chiamare una receive alla volta)
 - Using **guards**
 - introduced by Dijkstra in 1974
 - both in the synchronous and asynchronous case
 - mechanism used to realize **selective receive** using a *ricezione selettiva* ***select*** statements
 - e.g. sockets, Java NIO library,...

GUARDED COMMUNICATION STATEMENT

Comunicazione con Guardia

- **Guarded communication statement:**

$$B \ ; \ C \ \rightarrow \ S$$

Il concetto di "guardia" ha senso ed esprime la sua vera utilità quando abbiamo più guardie in parallelo (cioè più rami tra cui scegliere).

- **B is a boolean expression** (check sullo stato locale del processo)
 - if omitted, the value is true
 - **C is a communication statement (typically receive)** si può usare anche una send sincrona
 - **S is a statement block or list** il blocco di codice che si fa, se C è andata in porto (example, ricevuto un messaggio (receive) lo elaboro)
- **Together B and C comprise what is called the guard**
 - **A guard *succeeds*, if B is true and executing C would not cause a delay** (cioè se faccio la receive e c'è il messaggio, quindi non mi blocco)
Una guardia ha successo
 - **A guard *fails*, if B is false**
 - **A guard *blocks*, if C cannot yet be executed** (cioè se non c'è ancora il messaggio sul canale)

SELECTION + GUARDED COMMUNICATION

- With if statements:

[] (Operatore di alternativa):
Separa i diversi rami o guardie da valutare.

```
if B1;C1 → S1;  
[] B2;C2 → S2;  
[] B3;C3 → S3;  
fi
```

Aspetto il messaggio sui tre canali ed appena uno dei tre si sblocca (B e C) eseguo lo statement associato. Se arriva il messaggio su più di due canali, scelgo quale sbloccare in modo non-deterministico.

– Semantics della selezione non-deterministica

- first: evaluate the boolean expressions in the guards

Se tutte le guardie falliscono

- ~~if all fails~~, then the if terminates with no effect

Se almeno una guardia ha successo

- ~~if at least one succeeds~~, choose one of them **non-deterministically**

Se tutte le guardie aperte sono bloccanti, il processo si sospende in attesa finché almeno un canale associato a una guardia valida non diventa pronto.

- ~~if all guards block~~, then wait until one guard succeeds

- second: execute the C statement for the chosen guard
- third: execute the S block

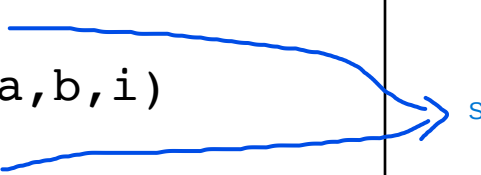
EXAMPLE

Ho una select su più rami

- Guarded command example:

Il server si mette in attesa sia sul canale
computeSum sia sul canale computeMul

```
if nReqs < max; receive computeSum(a,b,i)
  → nReqs+=1; send result[i](a+b)
[] nReqs < max; receive computeMul(a,b,i)
  → nReqs+=1; send result[i](a*b)
fi
```



LOOP + GUARDED COMMUNICATION

- With loop statements (represented by do statement):

```
do B1;C1 → S1;  
[ ] B2;C2 → S2;  
[ ] B3;C2 → S3;  
od
```

Condizione di Uscita: Se tutte le guardie booleane sono false (Bi per ogni ramo), il ciclo do ... od termina e il processo prosegue con l'istruzione successiva.



- in this case the selection process is repeated until all guards fail

Nell'esempio, ho un buffer che riceve caratteri in entrata fino a che è full e dopo li manda con una send sincrona, svuotando il buffer

```
process Copy(chan in(char), chan out(char)) {  
  char buffer[10];  
  int front = 0, rear = 0, count = 0;  
  do count < 10; receive in(buffer[rear])  
    → count++; rear = (rear+1)%10;  
  [ ] count > 0; send out(buffer[front])  
    → count--; front = (front+1)%10;  
  od  
}
```

indice dove andare a mettere il nuovo carattere che leggo

In una guardia, al posto del receive ci può essere un send sincrono (quindi bloccante)

Non-determinismo e reattività: Quando $0 < \text{count} < 10$, entrambe le condizioni booleane sono true. Il processo sceglierà, in modo non-deterministico, se leggere o scrivere in base a quale evento si verifica per primo (se arriva un nuovo dato su in o se il processo a valle è pronto a ricevere su out).

Devo avere delle condizioni booleane che mi dicono quando quel ramo della select è attivo perché non tutti i rami possono essere sempre attivi e quando arriva almeno un messaggio su un canale con ramo attivo, eseguo gli statements.

PRODUCERS / CONSUMERS

UN PROCESSO NON PUÒ MAI
BLOCCARE IL FLUSSO DI
CONTROLLO DI UN ALTRO PROCESSO

- Wrapping the bounded-buffer with a resource allocator process

Un altro esempio di Resource Allocator (qua, abbiamo che il BoundedBuffer è un Processo, quindi un Monitor attivo)

Un processo può bloccarsi se fa receive e non ci sono elementi su quel canale

```
process BoundedBufferManager {
  int nItems = 0;
  int maxElems = ...;
  Queue<ItemType> queue = ...;
  ItemType item;
  boolean ack = true;
  chan replyChan;

  do nItems < maxElems; receive put(item,replyChan)
    → queue.add(item);
    nItems++;
    send replyChan(ack);
  [] nItems > 0; receive get(replyChan)
    → ItemType el = queue.remove();
    nItems--;
    send replyChan(el);
  od
}
```

il Producer, facendo una send sul canale put, manda come messaggio l'item ed il canale in cui vuole ricevere l'ack. Il BBManager utilizzerà il canale mandato dal consumer per mandare l'ack

il Consumer, facendo una send sul canale get, manda come messaggio il canale in cui vuole ricevere l'item. Il BBManager utilizzerà il canale mandato dal consumer per mandare l'item

```
process Producer(chan myChan){
  boolean ack;

  loop {
    ItemType el = produce();
    send put(el,myChan);
    receive myChan(ack);
  }
}
```

Invia una put e poi si mette in attesa della risposta

```
process Consumer(chan myChan){

  loop {
    ItemType el;
    send get(myChan);
    receive myChan(el);
    consume(el);
  }
}
```

Invia una get e poi si mette in attesa della risposta

PEER-TO-PEER INTERACTION

- Decentralization of the responsibilities among the interacting parts (*peers*)
 - coordination by means of proper message-based interaction protocols among the peers
- An **example** A seconda del modello di interazione tra processi, cambia il modo di risolvere il problema (cioè come topologie diverse possono supportare proprietà diverse delle nostre soluzioni)
 - “The exchanging values problem”
 - N processes, each one with a local integer value v
 - the goal for every process is *to learn the smallest and the largest of the n local values* Dobbiamo coordinarli affinché tutti conoscano, alla fine del processo di coordinamento, qual'è il massimo ed il minimo
 - three different solution types Il problema può essere risolto organizzando i canali di comunicazione in tre modalità differenti, ciascuna con i propri punti di forza e debolezze (topologie diverse di scambio di messaggi)
 - 1 • **centralized**
 - 2 • **symmetric**
 - 3 • **ring**

1

CENTRALIZED SOLUTION

- One coordinator process receiving values from any other process and establishing the minimum and maximum

(il coordinatore si mette in attesa di ricevere tutti i messaggi con i valori di ogni processo. Dopo di che ogni processo si mette in attesa della risposta da coordinatore per sapere il minimo e massimo)

```
chan values(int), results[n](int smallest, int largest);
```

```
process P[0] { #coordinator
```

```
  int v = ....;
```

```
  int new, smallest = v, largest = v; # initial state
```

```
  for i in [ 1... n-1] {
```

```
    receive values(new);
```

```
    if (new < smallest)
```

```
      smallest = new;
```

```
    if (new > largest)
```

```
      largest = new;
```

```
  }
```

```
  # send the result to the other processes
```

```
  for i in [1...n-1]{
```

```
    send results[i](smallest, largest);
```

```
  }
```

```
}
```

```
process P[i]{
```

```
  int v = ..., smallest, largest;
```

```
  send values(v);
```

```
  receive results[i](smallest, largest);
```

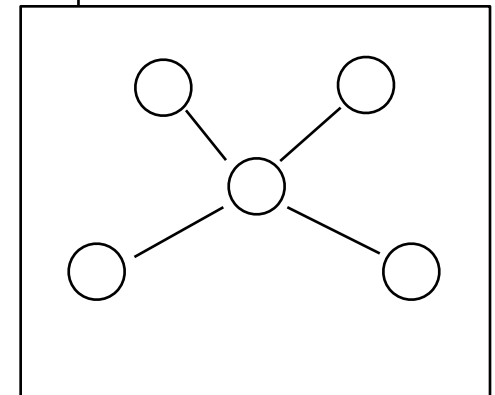
```
}
```

Ogni volta che riceve un messaggio, controlla ed aggiorna il minimo ed il massimo, se necessario. Il coordinatore sa a priori quanti messaggi riceverà

Se va giù il coordinatore?
Dobbiamo applicare un processo di elezione del nuovo coordinatore

Vantaggi/Svantaggi: Richiede pochi messaggi, ma il coordinatore rappresenta un potenziale collo di bottiglia (bottleneck) e un singolo punto di fallimento (Single Point of Failure).

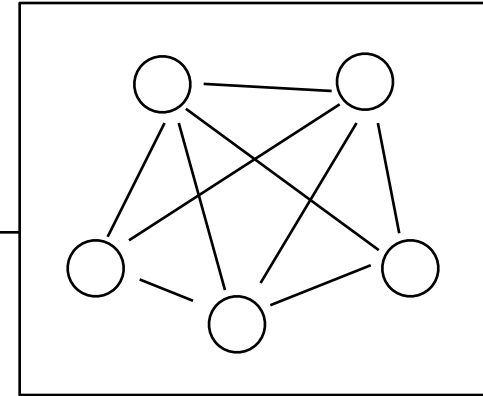
- small number of msgs numero di messaggi: $2*(n-1)$
- bottleneck with coordinator process
- coordinator's receive delayed



2 SYMMETRIC SOLUTION

Opposto del coordinatore: metto nelle condizioni tutti di poter decidere chi è il minimo e massimo

- Each process executes the same algorithm
 - first it sends its value to all the others
 - each process in parallel computes the min max



```
chan values[n](int);
```

```
process P[i = 0 to n - 1] {
  int v = ....;
  int new, smallest = v, largest = v; # initial state
  # send my values to other processes
  for j in [0...n-1], j != i {
    send values[j](v);
  }
  # gather values and save the smallest and largest
  for k in [1...n-1]{
    receive values[i](new);
    if (new < smallest)
      smallest = new;
    if (new > largest)
      largest = new;
  }
}
```

Ogni processo riceve da tutti gli altri i valori e calcola autonomamente il valore minimo e massimo

Vantaggi/Svantaggi: Altissima parallelizzazione ed eliminazione di ruoli speciali, ma richiede un numero molto elevato di messaggi

- large number of msgs $N*(N-1)$
- maximizing distribution
- concurrency

perché sparano tutte le send e poi ognuno fa le receive e calcola (a differenza degli altri che ci sono round e fasi da seguire che sono meno concorrenti)

3 RING SOLUTION

Sfruttiamo il fatto di creare una topologia (anello, ma non è detto che sia sempre questo) in cui propaghiamo la conoscenza che abbiamo a tutti

Fase A: Determinazione del Min/Max (Il primo giro)

L'obiettivo è far "viaggiare" i valori correnti lungo tutto l'anello affinché vengano confrontati con i valori locali di ogni processo. P[0] (il coordinatore) inizia inviando il proprio valore al successore. Ogni P[i] riceve la coppia (smallest, largest) dal predecessore, la confronta con il proprio valore locale v, aggiorna la coppia se necessario e la passa avanti. Quando il messaggio torna a P[0], il giro è completo. A questo punto, P[0] possiede i valori definitivi di minimo e massimo globale.

Fase B: Diffusione (Il secondo giro)

Il problema alla fine della Fase A è che solo P[0] conosce i risultati finali. P[0] invia nuovamente i valori definitivi nell'anello. Ogni P[i] riceve i valori aggiornati, li memorizza e li passa al successore senza ulteriori calcoli. Una volta che il messaggio torna a P[0], tutti i processi nell'anello sono sincronizzati.

- Organizing processes into a logical ring
 - each process P[i] receives msg from its predecessor and sends messages to its successor
 - 2 stages: determining the global min and max, spreading min and max

Ogni processo ha due elementi da cui ricevere ed inviare (invio al successivo, ricevo dal precedente). Mano a mano che interagiscono, arricchiscono la conoscenza

```
chan values(int), results[n](int smallest, int largest);
```

```
process P[0] { #coordinator il primo processo della catena
```

```
  int v = ....;
```

```
  int new, smallest = v, largest = v; # initial state
```

```
  [send values[1](smallest, largest);
```

```
  [receive values[0](smallest, largest);
```

```
  [send values[1](smallest, largest);
```

```
}
```

Faccio partire l'algo, mandando al successore il mio valore e, dopo aver fatto tutto il giro di tutto l'anello, ricevo il valore più piccolo ed il più grande dal precedente. Poi, essendo che solo il coordinatore conosce i risultati, manda al successivo, i valori aggiornati di massimo e minimo

```
process P[i = 1 to n - 1]{
```

```
  int v = ..., smallest, largest;
```

```
  [receive values[i](smallest, largest);
```

```
  if (v < smallest)
```

```
    smallest = v;
```

```
  if (v > largest)
```

```
    largest = v;
```

```
  [send values[(i+1)%n](smallest, largest);
```

```
  [receive values[i](smallest, largest);
```

```
  [send values[(i+1)%n](smallest, largest);
```

```
}
```

non ho un elemento di centralizzazione forte, ma uno debole

- small number of msgs

numero di messaggi: $2 \cdot (n-1)$

- limited concurrency

Prima di mandare la successore, ricevo dal precedente un valore che confronto con il mio e mando al successore il mio valore che ritengo minimo e massimo e ricevo quello dal precedente (il più piccolo ed il più grande, dato che ha fatto il giro di tutto l'anello). Poi, mando al successivo, i valori aggiornati di massimo e minimo

Vantaggi/Svantaggi:
Genera pochi messaggi, ma la concorrenza è limitata poiché i messaggi devono transitare sequenzialmente nodo per nodo.

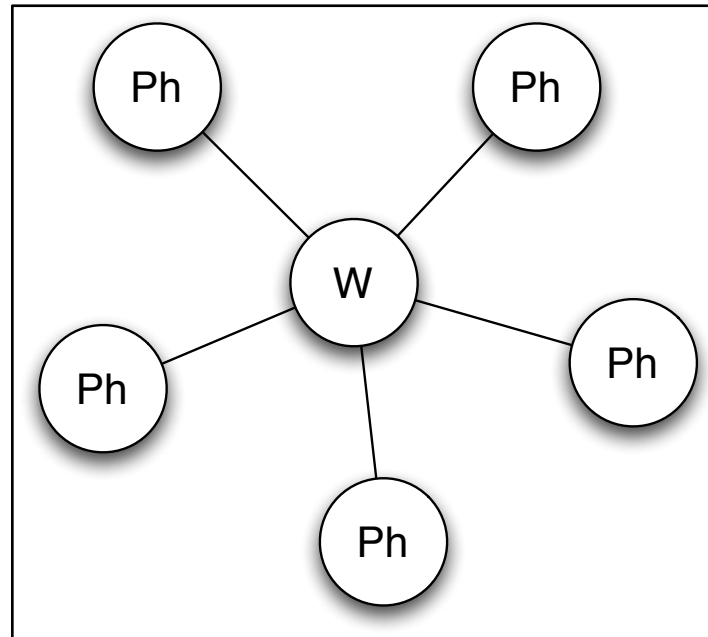
DINING PHILOSOPHERS

I canali diventano le forchette

- Dining philosopher problems in a distributed environment
 - each philosopher is running on a different network node
 - forks (resources) can be distributed as well
- General problem
 - resolving conflicts between processes in distributed systems

CENTRALIZED SOLUTION

- Using a *Waiter* process acting as resource (forks) allocator
 - responsible for deadlock avoidance and fairness



CENTRALIZED SOLUTION

```
chan getForks(int,int,chan);
chan releaseForks(int,int,chan);

process Waiter[i:0..N-1] {
  List<Request> pending = ...;
  boolean availForks[0..N-1] = {false,...};

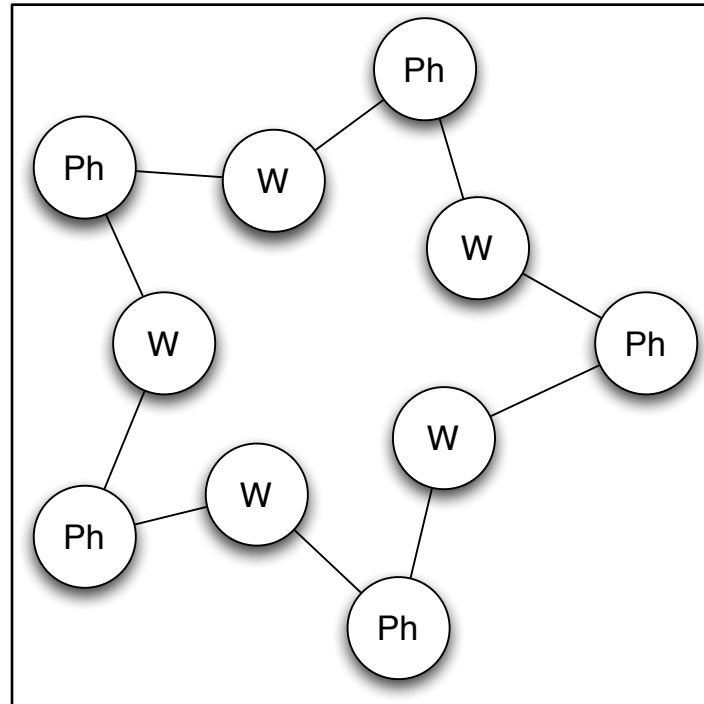
  do receive getForks(fork1,fork2, ReplyChanID)
    → if availForks[fork1] && availForks[fork2]
      availForks[fork1] = false;
      availForks[fork2] = false;
      send ReplyChanID(fork1,fork2)
    else
      pending.add(new Request(Reply,fork,fork2));
  [] receive releaseForks(fork1,fork2)
    →
      availForks[fork1] = true;
      availForks[fork2] = true;
      foreach Request r in pending
        if (availForks[r.fork1] && availForks[r.fork2])
          pending.remove(r);
          availForks[r.fork1] = true;
          availForks[r.fork2] = true;
          send req.ReplyChanID(req.fork1,req.fork2)
    od
}
```

```
process Philo[i:0..N-1,chan reply]{
  int first = i;
  int second = (i+1)%N;

  loop {
    think();
    send getForks(first,second,reply)
    receive reply(first,second)
    eat();
    send releaseForks(fork1,fork2)
  }
}
```

DISTRIBUTED SOLUTION

- Introducing a waiter managing each fork
 - a resource allocator for each fork



- Adopting a proper coordination protocol between philosophers and waiters to avoid deadlock
 - *resource hierarchy* strategy

DISTRIBUTED SOLUTION

```
chan getFork[0..N-1];
chan getForkReply[0..N-1];
chan releaseFork[0..N-1];
chan releaseForkReply[0..N-1];
```

```
process Waiter[i:0..N-1] {
  loop {
    receive getFork[i]();
    send getForkReply[i]();
    receive releaseFork[i]();
    send releaseForkReply[i]();
  }
}
```

```
process Philosopher[i:0..N-1] {
  int first = i; second = (i+1)%N;

  if (second < first){
    first = second;
    second = i;
  }

  loop {
    think();

    send getFork[first]();
    receive getForkReply[first]();
    send getFork[second]();
    receive getForkReply[second]();

    eat();

    send releaseFork[first]();
    receive releaseForkReply[first]();
    send releaseFork[second]();
    receive releaseForkReply[second]();
  }
}
```

SYNCHRONOUS MESSAGE PASSING

- **Synchronous** semantics
 - send blocked until the message has been received on the channel
 - no need of buffers

SENDER	RECEIVER
...	...
<code>p(i): send ch(msg)</code>	<code>q(j): receive ch(msg)</code>
...	...

- communication is atomic (inter)-action
 - the data transfer on a channel `ch` takes place only when control pointer of the sender is on the send instruction on `ch` and the control pointer of the receiver is on the receive instruction on `ch`

SYNCHRONOUS MESSAGE PASSING

- Abstract syntax frequently used
 - for send `ch(msg): ch ! msg`
 - for receive `ch(msg): ch ? msg`

PRODUCER-CONSUMER EXAMPLE

chan ch(int)	
PRODUCER	CONSUMER
x: integer	y: integer
loop forever:	loop forever:
p1: x ← produce	q1: receive ch(y)
p2: send ch(x)	q2: consume(y)

- With the synchronous semantics the data transfer takes place only when:
 - control pointer for P = p2
 - control pointer for Q = q1

SYNC COMMUNICATION: GO LANGUAGE

- Statically-typed language initially developed at Google in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson, also commonly referred to as "golang"
 - syntax loosely derived from that of C, adding garbage collection, type safety, some dynamic-typing capabilities, additional built-in types such as variable-length arrays and key-value maps, and a large standard library
 - nowadays quite widespread as a “system”, very fast language
- Built-in concurrency primitives: light-weight processes (**goroutines**), **channels**, and the **select** statement.
 - concurrency originates in Hoare's Communicating Sequential Processes (CSP),
 - "Do not communicate by sharing memory; instead, share memory by communicating"
 - ...but when needed it includes also support for mutex & classic sync mechanisms
- http://golang.org/doc/effective_go.html#sharing

è l'implementazione diretta della Guarded Communication (la Selective Receive). Permette a una goroutine di rimanere in attesa su più operazioni di canale contemporanea mente.

Go supporta sia memoria condivisa che non (ma il suo primo approccio era sui canali). Si possono creare programmi in cui si integra entrambi

Pronuncia: randevu

RENDEZ-VOUS

è una forma avanzata di comunicazione e sincronizzazione tra processi (communication scheme)

Non c'è trasferimento del flusso di controllo, ma c'è uno schema di sincronizzazione più articolato per gestire in modo più agevole quelle forme di interazione dove c'è una richiesta ed una risposta

- **Extended synchronization**

- with the sender waiting not only that the receiver would receive the request, but also the reply of a result
- The name *rendez-vous* invokes the image of two people who chose a place to meet
 - the first one to arrive must wait for the arrival of the second

- **Two roles**

assomiglia a una chiamata a procedura remota (RPC) sincrona: il mittente non si limita a consegnare un messaggio, ma resta congelato in attesa che il destinatario elabori la richiesta e gli restituisca un risultato.

- **calling processes**

Rappresenta il Client.

- process calling an *entry* on an accepting process
- needs to know the identity of the accepting process and the name of the entry
- **call** primitive

- **accepting process**

Rappresenta il Server.

- process accepting requests on specific *entries* and sending results with a reply
- need not to know the identity of the calling process



Permette al server di servire molti client diversi usando la stessa interfaccia, senza dover hardcodare i loro nomi nel codice.

CALL AND ACCEPT

A differenza di un semplice segnale di "messaggio ricevuto", il Rendez-vous nel modello Ada (o simili) prevede un'interazione in tre fasi:

- Sincronizzazione: Il chiamante invia la richiesta e attende.
- Elaborazione: Il ricevente accetta la richiesta ed esegue un blocco di codice (chiamato accept body).
- Risposta: Il ricevente invia il risultato e solo allora entrambi i processi tornano liberi di proseguire per conto proprio.

CLIENT	SERVER
<pre>integer parm, result loop forever p1: param ← ... p2: <u>call</u> server.service(parms,result) p3: use(result)</pre>	<pre>integer p, r loop forever q1: <u>accept</u> service(p,r) q2: <do the service with p> qX: ... qN: r ← <result></pre>

Non è trasferimento di controllo: è una send che sotto ha una receive della risposta (viene inviato un messaggio al server ed il client si blocca aspettando la risposta da server). La receive specifica di una call si chiama accept: la call viene ricevuta dal server quando fa accept, computa e poi manda la risposta (solo quando riceve risposta, il client si sblocca)

Chiusura dell'Accept: L'ambiente di esecuzione (supporto software di basso livello fornito dal linguaggio di programmazione) trasferisce automaticamente il valore contenuto in "r" nella variabile result del Client.

RENDEZ-VOUS IN ADA

- Direct support by means of *entry* call
 - basic communication mechanism among tasks
- Accept statement
 - entries declared in the task public interface
 - entries implemented in the task body by means of accept statement

Calling Task	Called Task
	<code>task T;</code>
	<code> entry E(formals);</code>
	<code> : -- declaration</code>
	<code>begin</code>
<code>:</code>	<code> :</code>
<code>T.E(actuals) -- calls the</code>	<code> accept E(formals);</code>
<code> rendezvous</code>	<code> : -- body of rendezvous</code>
	<code> end accept;</code>
	<code> :</code>
	<code>end;</code>

EXAMPLE

```
with Ada.Text_IO;
use Ada.Text_IO;

procedure HotDog is

  task Gourmet is
    entry Make_A_Hot_Dog;
  end Gourmet;

  task body Gourmet is
  begin
    Put_Line("I am ready to make a hot dog for you");
    for Index in 1..4 loop
      accept Make_A_Hot_Dog do
        delay 0.8;
        Put("Put hot dog in bun ");
        Put_Line("and add mustard");
      end Make_A_Hot_Dog;
    end loop;
    Put_Line("I am out of hot dogs");
  end Gourmet;

begin
  for Index in 1..4 loop
    Gourmet.Make_A_Hot_Dog;
    delay 0.1;
    Put_Line("Eat the resulting hot dog");
    New_Line;
  end loop;
  Put_Line("I am not hungry any longer");
end HotDog;
```

-- Result of execution

I am ready to make a hot dog for you
Put hot dog in bun and add mustard
Eat the resulting hot dog

Put hot dog in bun and add mustard
Eat the resulting hot dog

Put hot dog in bun and add mustard
Eat the resulting hot dog

Put hot dog in bun and add mustard
I am out of hot dogs
Eat the resulting hot dog

I am not hungry any longer

BIBLIOGRAPHY

- **[BEN-ARI]** “Principle of Concurrent and Distributed Programming”, Ben-Ari, Addison-Wesley
- **[AND]** “Foundation of Multithreaded, Parallel and Distributed Programming”, Gregory R. Andrews, Addison Wesley
- Dijkstra, Edsger W. "EWD472: Guarded commands, non-determinacy and formal. derivation of programs."